

Programação Orientada a Objetos

Resumo Aprofundado com Exemplos Completos

Prof. Rodrigo Martins Pagliares · Prova: 20 questões de múltipla escolha

Tópico	Questões	% da Prova
■ Princípios GRASP	5	25%
■ Coleções em Java	5	25%
■ Exceções em Java	3	15%
■ Diagrama de Sequência do Sistema	2	10%
■ Modelos de Análise e Design OO	2	10%
■ Contrato de Operação	1	5%
■ Generics	1	5%
■ Design para Visibilidade	1	5%

■ Princípios GRASP — 5 questões (25%)

GRASP significa **General Responsibility Assignment Software Patterns** — padrões gerais para atribuir responsabilidades a classes e objetos. A ideia central é simples: quando você cria um sistema OO, precisa decidir **qual classe faz o quê**. Os 9 princípios GRASP te dão critérios para tomar essas decisões de forma que o sistema fique fácil de entender, manter e modificar.

■ Analogia do mundo real

Imagine uma empresa. Você precisa decidir quem é responsável por cada tarefa: quem assina contratos, quem compra suprimentos, quem atende clientes. GRASP faz o mesmo, mas para classes de software.

1. Information Expert — o princípio mais importante

Definição: Atribua a responsabilidade de realizar uma tarefa à classe que possui (ou pode calcular) as informações necessárias para cumpri-la.

Em outras palavras: **quem tem os dados, faz o trabalho.**

■ Analogia

Se você quer saber o total de uma nota fiscal, quem melhor para calcular do que a própria nota fiscal? Ela tem todos os itens, quantidades e preços. Pedir para o caixa calcular isso seria estranho — ele não tem os dados detalhados.

■ Exemplo: Sistema de Hostel — quem calcula a taxa de ocupação?

Imagine as classes: **Hostel**, **Quarto**, **Reserva**, **Recepcionista**.

Pergunta: qual classe deve ter o método `calcularTaxaOcupacao()`?

- Recepcionista? — Não. Ela não tem a lista de quartos/reservas diretamente.
- Reserva? — Não. Uma reserva só sabe do próprio quarto, não de todos.
- Hostel? — **SIM!** Hostel possui a lista de todos os quartos e todas as reservas.

```
class Hostel {  
    private List<Quarto> quartos; // TEM os quartos  
    private List<Reserva> reservas; // TEM as reservas  
  
    public double calcularTaxaOcupacao() {  
        long ocupados = quartos.stream()  
            .filter(q -> q.estaOcupado()).count();  
        return (double) ocupados / quartos.size() * 100;  
    }  
}
```

■ Armadilhas e erros comuns:

Atribuir o cálculo a uma classe que **NÃO** possui os dados — ela precisaria buscar as informações de outras classes, aumentando acoplamento.

Confundir Information Expert com a classe que 'parece mais importante'. O critério é quem **TEM** os dados.

2. Creator — quem deve criar objetos?

Definição: A classe B deve ser responsável por criar instâncias da classe A se pelo menos uma das condições abaixo for verdadeira:

- ✓ B **agrega** ou **contém** objetos do tipo A (relação todo-parte)
- ✓ B **registra** ou rastreia objetos do tipo A
- ✓ B **usa** objetos do tipo A de forma próxima
- ✓ B possui os **dados de inicialização** de A

■ Analogia

Quem monta um pedido no iFood? O próprio Pedido — ele sabe quais itens, quantidades e preços. Não faz sentido pedir pro cliente que monte os itens do pedido separadamente fora do pedido.

■ Exemplo: Venda cria LinhaDeItemDeVenda

Temos: **Venda**, **LinhaDeItemDeVenda**, **Produto**, **Caixa**.

Pergunta: quem deve criar uma **LinhaDeItemDeVenda** quando o caixa adiciona um item?

- Caixa? — Não. Caixa não agrega linhas de item.
- Produto? — Não. Produto não contém linhas de venda.
- Venda? — **SIM!** Venda agrega LinhasDeItem (contém N delas) e tem os dados para iniciá-las.

```
class Venda {  
    private List<LinhaDeItemDeVenda> itens = new ArrayList<>();  
  
    public void adicionarItem(Produto produto, int qtd) {  
        // Venda CRIA a linha — ela tem os dados necessários  
        LinhaDeItemDeVenda linha = new LinhaDeItemDeVenda(produto, qtd);  
        itens.add(linha);  
    }  
}
```

■■ Armadilhas e erros comuns:

Deixar a classe Caixa (ou qualquer classe que não agrega A) criar objetos do tipo A — isso aumenta acoplamento desnecessariamente.

Na prova: quando aparecer 'quem deve criar X?', veja quem **CONTÉM** ou **AGREGA** X.

3. Controller — quem recebe os eventos do sistema?

Definição: Atribuir a responsabilidade de receber e coordenar (controlar) eventos de sistema a um objeto que representa:

- O **sistema como um todo** (fachada — representa o sistema inteiro)
- Um **caso de uso específico** (controlador de sessão/caso de uso)

■ Analogia

Num restaurante, o garçom recebe o pedido do cliente e repassa para a cozinha. Ele é o Controller: não cozinha, não faz a comida — apenas coordena. A cozinha (domínio) faz o trabalho real.

■ Exemplo: Sistema de Hostel — quem recebe o evento 'registrarReserva'?

A interface (UI) captura o clique do usuário. Ela NÃO deve processar a reserva.

O Controller recebe o evento e delega para o domínio.

// ERRADO — lógica de negócio dentro da UI

```
class TelaReserva {  
    void btnConfirmarClicado() {  
        Reserva r = new Reserva(quarto, hospede, data);  
        hostel.reservas.add(r); // UI acessando domínio diretamente!  
    }  
}
```

// CORRETO — Controller como intermediário

```
class GerenciadorDeReservas { // Controller (caso de uso)  
    public void registrarReserva(Quarto q, Hospede h, LocalDate d) {  
        Reserva r = hostel.criarReserva(q, h, d); // delega ao domínio  
    }  
}
```

■■ Armadilhas e erros comuns:

Controller NÃO é um objeto de UI (tela, botão, formulário).

Controller NÃO faz a lógica de negócio — ele delega para as classes de domínio.

Confundir Controller GRASP com o 'C' do padrão MVC (são relacionados, mas não idênticos).

4. Low Coupling e 5. High Cohesion — a dupla fundamental

Low Coupling (Baixo Acoplamento)

Acoplamento = quanto uma classe DEPENDE de outras. Alto acoplamento = mudança numa classe quebra outras.

Meta: minimizar dependências.

// Alto acoplamento — ruim

```
class Reserva {  
    Quarto q; Hospede h;  
    Pagamento pg; Email em;  
    Relatorio rel; Log log;  
    // depende de tudo!  
}
```

High Cohesion (Alta Coesão)

Coesão = quão relacionadas são as responsabilidades de uma classe. Baixa coesão = classe faz coisas demais.

Meta: cada classe faz coisas relacionadas.

// Baixa coesão — ruim

```
class Hostel {  
    void fazerReserva() {...}  
    void enviarEmail() {...}  
    void gerarRelatorio() {...}  
    void procesarPagto() {...}  
    // faz tudo!  
}
```

■ Analogia

Pense num canivete suíço (baixa coesão, alto acoplamento de funções) vs. uma faca de chef (alta coesão — faz uma coisa, mas faz bem). Uma classe coesa e desacoplada é como a faca de chef: especializada e fácil de trocar.

6. Polymorphism — elimine if/else baseado em tipo

Definição: Quando o comportamento varia de acordo com o tipo do objeto, use polimorfismo (herança + @Override) em vez de verificação de tipo com if/else.

■ Exemplo: Tipos de pagamento em um hostel

// SEM polimorfismo — ruim, frágil

```
void processar(Pagamento p) {  
    if (p.tipo.equals("dinheiro")) { ... }  
    else if (p.tipo.equals("cartao")) { ... }  
    else if (p.tipo.equals("pix")) { ... } // adicionar novo tipo = modificar aqui  
}
```

// COM polimorfismo — correto

```
abstract class Pagamento {  
    abstract void processar(); // cada subclasse implementa do seu jeito  
}
```

```
class PagamentoDinheiro extends Pagamento {
```

```
@Override void processar() { /* conta o dinheiro */ }
```

```
}
```

```
class PagamentoCartao extends Pagamento {
```

```
@Override void processar() { /* lê o cartão */ }
```

```
}
```

// adicionar Pix = nova classe, sem tocar no código existente

7. Pure Fabrication — classes que não existem no domínio real

Definição: Criar uma classe que NÃO representa nenhum conceito do mundo real, mas que existe apenas para manter alta coesão e baixo acoplamento no software.

■ Analogia

No mundo real, um 'Repositório de Reservas' não existe — hóspedes fazem reservas num hostel, ponto final. Mas no software, criar uma classe RepositorioDeReserva para isolar o acesso ao banco de dados faz todo sentido — ela cuida só disso, mantendo Hostel coeso.

■ Exemplo: RepositorioDeReserva — classe artificial para organizar acesso a dados

// Pure Fabrication — não existe no domínio, mas organiza o software

```
class RepositorioDeReserva {  
    private Map<String, Reserva> reservas = new LinkedHashMap<>();  
  
    public void salvar(Reserva r) { reservas.put(r.getId(), r); }  
    public Reserva buscar(String id) { return reservas.get(id); }  
    public List<Reserva> listarTodas() { return new ArrayList<>(reservas.values()); }
```

```
}
```

Sem essa classe, Hostel precisaria fazer tudo isso — ficaria enorme e com baixa coesão.

8. Indirection e 9. Protected Variations

8. Indirection

Introduzir um objeto intermediário para desacoplar dois elementos que não devem depender diretamente um do outro.

Exemplo: UI → **Controller** → Domínio

O Controller é o intermediário que desacopla UI do domínio.

Suporta Low Coupling — se UI mudar, domínio não muda.

Toda Pure Fabrication é uma forma de Indirection.

9. Protected Variations

Identificar pontos de variação e criar uma interface estável ao redor deles.

Exemplo: a forma de pagamento pode variar (dinheiro, cartão, pix). Encapsula com interface [Pagamento](#).

Mudança interna não afeta quem usa a interface.

Usa Polymorphism internamente.

Base do princípio Open/Closed (SOLID).

Relações entre os 9 princípios GRASP:

- Information Expert e Creator frequentemente apontam para a mesma classe
- Indirection → suporta Low Coupling
- Pure Fabrication → aumenta High Cohesion e Low Coupling (é um tipo de Indirection)
- Protected Variations → usa Polymorphism internamente
- Low Coupling e High Cohesion estão sempre em tensão — equilibre nas decisões de design

■ Coleções em Java — 5 questões (25%)

Coleções são estruturas de dados prontas que o Java oferece. Em vez de gerenciar arrays manualmente, você usa uma coleção adequada ao problema. A prova testa: hierarquia de interfaces, características de cada implementação, e as diferenças conceituais entre ordered/sorted.

A hierarquia — de onde vem cada classe?

Tudo começa com a interface **Iterable** (que permite usar for-each). Dela herda **Collection**. De **Collection** herdam **List**, **Set** e **Queue**. **Map** é separado — NÃO herda de **Collection**.

Iterable → **Collection** → List, Set, Queue

Map NÃO herda de Collection — essa é a cilada mais cobrada!

List — quando a ordem e índice importam

■ Analogia

Uma fila de cinema: as pessoas estão em ordem, você sabe quem é o 3º da fila, e a mesma pessoa pode aparecer duas vezes (se comprou dois ingressos). List funciona assim.

■ Exemplo: ArrayList vs LinkedList

// ArrayList: melhor para acesso por posição

```
List<String> nomes = new ArrayList<>();
```

```
nomes.add("Pedro"); // índice 0
```

```
nomes.add("Julia"); // índice 1
```

```
nomes.add("Aurelio"); // índice 2
```

```
System.out.println(nomes.get(1)); // imprime: Julia — O(1)
```

// LinkedList: melhor para inserção/remoção frequente no início/meio

```
List<String> fila = new LinkedList<>();
```

```
fila.add(0, "prioritario"); // insere no início — O(1)
```

ArrayList mantém a ordem de iteração pelo índice — afirmar que não mantém é INCORRETO! (questão 4.4a da prova2)

Set — quando duplicatas não são permitidas

■ Analogia

Um conjunto de CPFs cadastrados: o mesmo CPF não pode aparecer duas vezes. Se você tentar adicionar um CPF já existente, simplesmente nada acontece — o método `add()` retorna `false`.

■ Exemplo: HashSet vs LinkedHashSet vs TreeSet — qual usar?

Cenário 1: armazenar IDs de hóspedes, sem importar a ordem:

```
Set<String> ids = new HashSet<>(); // unordered, mais rápido
```

```
ids.add("H001"); ids.add("H002"); ids.add("H001"); // duplicata ignorada
```

```
System.out.println(ids.size()); // 2 — não 3
```

Cenário 2: listar hóspedes na ordem em que chegaram:

```
Set<String> chegada = new LinkedHashSet<>(); // ordered (inserção)
```

```
chegada.add("Ana"); chegada.add("Bia"); chegada.add("Carlos");
```

```
// iteração sempre: Ana → Bia → Carlos
```

Cenário 3: listar hóspedes em ordem alfabética:

```
Set<String> alfa = new TreeSet<>(); // sorted (ordem natural)
```

```
alfa.add("Carlos"); alfa.add("Ana"); alfa.add("Bia");
```

```
// iteração sempre: Ana → Bia → Carlos (ordenado!)
```

ordered vs sorted — diferença fundamental:

- **ordered:** a ordem de iteração é previsível. Ex: `LinkedHashSet` itera na ordem em que os elementos foram inseridos.
- **sorted:** os elementos são ordenados por valor (a < b, ordem alfabética, etc). Ex: `TreeSet` ordena pelo `Comparable` do objeto.
- Sorted implica ordered. Ordered NÃO implica sorted.
- **HashSet é unordered** — a ordem de iteração é imprevisível e pode mudar.

Map — chave → valor, e por que não é uma Collection

■ Analogia

Um dicionário: você busca uma palavra (chave) e encontra a definição (valor). Cada palavra é única. Map funciona assim — chaves únicas mapeadas para valores.

■ Exemplo: HashMap com hóspedes de um hostel

```
// Map: CPF (chave) → Hospede (valor)
```

```
Map<String, Hospede> cadastro = new HashMap<>();
```

```
cadastro.put("123.456.789-00", new Hospede("Pedro", "Silva"));
```

```
cadastro.put("987.654.321-00", new Hospede("Julia", "Santos"));
```

```
// Buscar por chave — O(1)
```

```
Hospede h = cadastro.get("123.456.789-00");
```

```
// Verificar se existe
```

```
if (cadastro.containsKey("123.456.789-00")) { ... }
```

```
// Iterar por todos
```

```
for (Map.Entry<String, Hospede> entry : cadastro.entrySet()) {
```

```
    System.out.println(entry.getKey() + " → " + entry.getValue().getNome());
```

```
}
```

Por que Map não herda de Collection? Collection representa um grupo de elementos individuais. Map representa pares chave-valor — uma estrutura diferente que não se encaixa na hierarquia de Collection.

Implementação	Ordered?	Sorted?	Duplicatas?	Complexidade
ArrayList	✓ (índice)	✗	✓	add O(1)amort, get O(1)

LinkedList	✓ (inserção)	✗	✓	add O(1), get O(n)
HashSet	✗ unordered	✗	✗	add/contains O(1)
LinkedHashSet	✓ (inserção)	✗	✗	add/contains O(1)
TreeSet	✓	✓	✗	add/contains O(log n)
HashMap	✗ unordered	✗	chaves ✗	put/get O(1)
LinkedHashMap	✓ (inserção)	✗	chaves ✗	put/get O(1)
TreeMap	✓	✓	chaves ✗	put/get O(log n)

■ Exceções em Java — 3 questões (15%)

Exceções são o mecanismo do Java para lidar com situações anormais durante a execução. Em vez de retornar um código de erro (como -1 ou null), o Java lança um objeto de exceção que interrompe o fluxo normal e pode ser capturado por quem chamou o método.

Hierarquia — de onde vem cada exceção?

■ Analogia

Pense em problemas num hostel. Erro do sistema elétrico (não dá pra tratar — Error). Quarto não disponível (previsível, deve tratar — checked). Tentar acessar uma lista vazia por engano de programação (bug — unchecked/RuntimeException).

Throwable (tudo que pode ser lançado)

■■■■ Error — problema grave da JVM. NUNCA capture.

■ Ex: OutOfMemoryError, StackOverflowError

■■■■ Exception — situações que podem ser tratadas

■■■■ RuntimeException → UNCHECKED

■ Indica bug no código. Não obrigatório declarar.

■ Ex: NullPointerException, ArrayIndexOutOfBoundsException

■■■■ Demais subclasses de Exception → CHECKED

Situação prevista que pode falhar. Obrigatório tratar.

Ex: IOException, SQLException, suas exceções customizadas

Checked (verificada):

- O compilador EXIGE que você declare throws ou use try/catch
- Representa: 'isso pode falhar por algo fora do seu controle'
- Ex: arquivo não encontrado, banco de dados indisponível

Unchecked (não verificada):

- O compilador NÃO exige tratamento
- Representa: 'isso é um bug — deveria ter verificado antes'
- Ex: índice inválido, ponteiro nulo, divisão por zero

INCORRETO: 'exceções verificadas não precisam ser especificadas na assinatura dos métodos' ← errado!
(questão 4.3e da prova2)

Blocos try / catch / finally — como funcionam

■ Exemplo: Fluxo completo com todos os blocos

```
try {  
    // código que pode lançar exceção  
  
    Hospede h = hostel.buscarHospede("H001"); // pode lançar ExcecaoNaoEncontrado  
  
    System.out.println("Hóspede: " + h.getNome()); // só executa se não lançou
```

```

}

catch (ExcecaoHospedeNaoEncontrado e) {
    // executa SOMENTE se a exceção foi lançada
    System.out.println("Erro: " + e.getMessage());
}

finally {
    // SEMPRE executa — com ou sem exceção
    System.out.println("Operação concluída (sucesso ou falha)");
}

```

finally com return sobrescreve o try:

```
try { return 1; } finally { return 2; } // retorna 2 — finally vence!
```

Multicatch (Java 7+) — capturar vários tipos de uma vez:

```

catch (IOException | SQLException e) { // um catch para dois tipos

    System.out.println("Erro de I/O ou banco: " + e.getMessage());

}

```

Criando exceções customizadas — passo a passo

A prova cobrou exatamente esse padrão. Entenda o porquê de cada linha:

■ Exemplo: ExcecaoHospedeNaoEncontrado — linha a linha

// 1. Herdar de Exception torna esta uma exceção CHECKED

```
class ExcecaoHospedeNaoEncontrado extends Exception { // 1.1
```

// 2. Atributos próprios desta exceção (informação extra)

```
private String nome; // 1.2
```

```
private String sobrenome; // 1.2
```

// 3. Construtor que recebe tudo necessário

```
public ExcecaoHospedeNaoEncontrado(String mensagem, String nome, String sobrenome) {
```

```
    super(mensagem); // 1.3 — passa a msg para Exception, que a guarda em 'message'
```

```
    this.nome = nome; // guarda info extra
```

```
    this.sobrenome = sobrenome; // guarda info extra
```

```
}
```

```
}
```

Por que super(mensagem)? A classe Exception (que herda de Throwable) tem um atributo privado chamado `message`. O único jeito de inicializá-lo é chamando `super()`. Depois, `getMessage()` retorna esse valor.

■ Exemplo: Usando a exceção no método de busca — 1.4

// Dentro da classe Albergue:

```
public Hospede pesquisarHospedePeloNome(String nome, String sobrenome)
```

```
throws ExcecaoHospedeNaoEncontrado { // declara que pode lançar

for (Hospede h : hospedes) {
    if (h.getNome().equals(nome) && h.getSobrenome().contains(sobrenome)) {
        return h; // encontrou — retorna normalmente
    }
}

// 1.4 — não encontrou: lança a exceção com informações detalhadas
throw new ExcecaoHospedeNaoEncontrado("Hospede nao encontrado", nome, sobrenome);
}
```

■ Exemplo: Capturando no método main — 1.5 e 1.6

```
Hospede c = null;

try { // 1.5
    c = hl.pesquisarHospedePeloNome("Florentino", "Ariza");
    System.out.println("Encontrado: " + c.getNome());
} catch (ExcecaoHospedeNaoEncontrado e) { // 1.6
    System.out.println("Hospede nao encontrado: " + e.getMessage());
}
```

■ DSS · Contrato de Operação · Modelos OO

De onde surgem os artefatos? — O fluxo completo do PU

O Processo Unificado (PU) é uma metodologia de desenvolvimento. Cada artefato surge a partir do anterior:

- 1■■■ **Caso de Uso** — descreve o que o usuário quer fazer em linguagem natural
↓ Lemos o caso de uso e identificamos as operações de sistema
- 2■■■ **Modelo de Domínio** — classes conceituais do mundo real (análise)
↓ Transformamos os cenários em interações ator-sistema
- 3■■■ **DSS (Diagrama de Sequência do Sistema)** — operações de sistema (análise)
↓ Especificamos formalmente o que cada operação faz
- 4■■■ **Contrato de Operação** — pré/pós-condições de cada operação
↓ Decidimos quais objetos colaboram para implementar
- 5■■■ **DS (Diagrama de Sequência de design)** — objetos internos colaborando
↓ Traduzimos o design para código
- 6■■■ **Código Java**

DSS vs DS — a diferença que mais cai na prova

DSS — Sistema como CAIXA PRETA

Fase: Análise / Requisitos

O sistema é tratado como uma caixa preta. Não nos importa como ele faz — só o que entra e o que sai.

Participantes: **ator externo + :Sistema**

Mostra: **operações de sistema** (eventos externos)

Exemplo: o caixa clica 'iniciar venda' → sistema registra → sistema exibe tela de itens.

DS — Sistema por DENTRO

Fase: Design / Projeto

Abrimos a caixa preta. Vemos quais objetos internos colaboram para implementar cada operação.

Participantes: **objetos internos** (Registradora, Venda, CatalogoProdutos...)

Mostra: **mensagens entre objetos**

Exemplo: Registradora.entrarItem() → Catalogo.buscarEspec() → Venda.adicionarItem().

■■■ Armadilhas e erros comuns:

DSS e DS SÃO diferentes — embora ambos usem notação UML de sequência.

DSS pertence à ANÁLISE. DS pertence ao DESIGN. NÃO inverte.

Como construir um DSS — tríades e operações de sistema

Para extrair as operações de sistema do caso de uso, usamos o estilo de escrita em **tríades**:

Formato da tríade:

1. Sistema solicita [informação]
2. **Usuário informa [dado]** → gera a operação de sistema
3. Sistema responde [resultado]

Regra: N° de tríades = N° de operações de sistema

O passo 'Usuário informa' sempre gera uma operação de sistema.

■ Exemplo: Caso de uso Processar Venda — extraindo operações

Tríade 1:

Sistema exibe a opção de nova venda

Usuário seleciona nova venda → `criarNovaVenda()`

Sistema cria e exibe a venda vazia

Tríade 2 (dentro do loop — enquanto há itens):

Sistema solicita identificador e quantidade

Usuário informa o identificador e quantidade → `entrarItem(idItem, qtd)`

Sistema exibe descrição, preço e total parcial

Tríade 3:

Sistema fornece opção de finalizar

Usuário confirma finalização → `finalizarVenda()`

Sistema exibe total com impostos

Tríade 4:

Sistema solicita informação de pagamento

Usuário informa o valor → `fazerPagamento(quantia)`

Sistema exibe troco e recibo

Regras de nomenclatura de operações:

- ✓ Iniciar com verbo genérico: entrar, criar, finalizar, calcular
- ✓ Parâmetros = exatamente os dados que o usuário informou
- ✓ Bool → forma de pergunta: `foiVendaFinalizada()`
- ✗ Verbos que sugerem tecnologia: `escanear()`, `swipe()`, `clicar()`
- ✗ Verbos vagos: `processarX()`, `tratarX()`, `fazerX()`, `handleX()`
- ✗ Padrão JavaBeans: `setItem()`, `getItem()`
- ✗ Repetir o mesmo parâmetro em mais de uma operação do caso de uso

Contrato de Operação — o que é e por que existe

O DSS define O QUE o sistema faz (as operações). O Contrato especifica formalmente o comportamento esperado de cada operação — serve de contrato entre a análise e o design.

■ Exemplo: Contrato completo para `entrarItem(idItem, qtd)`

Operação: `entrarItem(idItem: ItemID, qtd: Inteiro)`

Responsabilidades: Registrar um item da venda e atualizar o total parcial.

Pré-condições: Uma venda está em andamento (foi criada por `criarNovaVenda()`).

Pós-condições (voz passada — estado APÓS a execução):

- Uma instância de `LinhaDeltemDeVenda` **foi criada**.
- A `LinhaDeltemDeVenda` **foi associada** à Venda corrente.
- A quantidade da `LinhaDeltemDeVenda` **foi definida** como qtd.
- O total parcial da Venda **foi recalculado**.

■ ■ Armadilhas e erros comuns:

Pós-condições são sempre descritas no PASSADO (foi criada, foi associada, foi modificado).

Pré-condições são o que se assume verdadeiro ANTES — não o que o método verifica.

Responsabilidades descrevem O QUE faz, não COMO faz — não inclua implementação.

Modelos de Análise e Design OO — a diferença essencial

Análise OO — WHAT

Pergunta: o que existe no problema?

Produto: **Modelo de Domínio**

- Classes conceituais do mundo real
- Substantivos do domínio viram classes
- Normalmente SEM métodos
- Se X é número ou texto → atributo, não classe!

Ex: Hostel, Quarto, Hospede, Reserva

Design OO — HOW

Pergunta: como o software vai funcionar?

Produto: **Modelo de Projeto** (DS, código)

- Classes de software
- Com métodos, visibilidade, padrões
- Controladores, repositórios, fachadas
- Gerado a partir dos Contratos

Ex: GerenciadorDeReservas,
RepositorioDeReserva

■ ■ Armadilhas e erros comuns:

4.8e: 'Se X é número no mundo real, X provavelmente é classe conceitual' ← ERRADO. Se X é número ou texto, é ATRIBUTO.

4.9c: 'Modelo de Domínio engloba conceitos que descrevem objetos de software' ← ERRADO. MD descreve o mundo real.

4.9e: 'É mais importante saber projetar objetos do que entender UML' ← ERRADO.

Análise e Design NÃO são sinônimos e não podem ser usados de forma intercambiável.

Generics — 1 questão

Generics permitem criar classes e métodos que funcionam com qualquer tipo, mantendo a segurança de tipo em tempo de **compilação**.

■ Exemplo: Por que Generics existem? — o problema sem eles

```
// SEM Generics — Collection de Object  
  
List reservas = new ArrayList(); // aceita qualquer Object  
  
reservas.add(new Reserva("101", hospede)); // ok  
reservas.add("string errada"); // compilador não reclama!  
  
// Na hora de usar, precisa de cast — pode explodir em runtime  
Reserva r = (Reserva) reservas.get(1); // ClassCastException em runtime!  
  
// COM Generics — seguro  
  
List<Reserva> reservas = new ArrayList<>(); // só aceita Reserva  
  
reservas.add(new Reserva("101", hospede)); // ok  
reservas.add("string errada"); // ERRO DE COMPILAÇÃO — capturado antes!  
  
Reserva r = reservas.get(0); // sem cast — compilador sabe o tipo
```

Regras de Generics que caem na prova:

- ✓ Não aceita tipos primitivos: use Integer, Double, Boolean (não int, double, boolean)
- ✓ Checagem de tipo ocorre em TEMPO DE COMPILAÇÃO
- ✓ Elimina casts desnecessários
- ✓ Operador diamante <> infere o tipo: new ArrayList<>()

TYPE ERASURE: em Java, a informação do tipo genérico é APAGADA em tempo de execução.

A JVM não sabe que era List — ela vê apenas List. Por isso, INCORRETO afirmar que 'a definição do tipo genérico é mantida em runtime'. ← questão 4.1d

Design para Visibilidade — 1 questão

Para que o objeto A possa enviar uma mensagem ao objeto B (chamar um método de B), A precisa ter **visibilidade** de B — ou seja, A precisa 'enxergar' B. Há 4 tipos:

■ Exemplo: Os 4 tipos de visibilidade — com código

1. **Visibilidade de ATRIBUTO** (persistente — disponível em todos os métodos)

```
class Registradora {  
  
    private Venda vendaAtual; // atributo — B é campo de A  
  
    public void finalizarVenda() {  
  
        vendaAtual.calcularTotal(); // pode chamar pois vendaAtual é atributo  
  
    }  
}
```



```
}
```

2. Visibilidade de PARÂMETRO (temporária — só dura enquanto o método executa)

```
public void processar(Venda v) { // B recebido como parâmetro
    v.calcularTotal(); // pode chamar pois v foi passado como argumento
}
```

3. Visibilidade LOCAL (temporária — só dentro do método)

```
public void criarVenda() {
    Venda v = new Venda(); // B declarado localmente dentro do método
    v.inicializar();
    this.vendaAtual = v; // pode promover para atributo se necessário
}
```

4. Visibilidade GLOBAL (persistente — acessível de qualquer lugar)

```
class Config {
    public static final Hostel INSTANCIA = new Hostel(); // singleton/global
}
```

Raro em bom design OO — viola Low Coupling.

Datas em Java — antes e depois do Java 8

Pré-Java 8 (legado):

Date: data e hora básicas

GregorianCalendar: calendário completo

DateFormat: formatação de datas

```
// Criar data atual
```

```
Date agora = new Date();
```

```
// Criar data específica
```

```
Calendar c = new GregorianCalendar(2026,5,22);
```

```
// Somar dias
```

```
c.add(Calendar.DAY_OF_MONTH, 3);
```

Cilada 4.6e: Calendar herda de GregorianCalendar
← ERRADO!

É o inverso: GregorianCalendar estende Calendar.

Java 8+ (moderno — preferir sempre):

LocalDate: só data (sem hora)

LocalTime: só hora

LocalDateTime: data e hora

DateTimeFormatter: formatação

```
// Data e hora atuais
```

```
LocalDateTime agora = LocalDateTime.now();
```

```
// Data específica
```

```
LocalDate d = LocalDate.of(2026, 6, 22);
```

```
// Somar dias (NÃO é .add()!)
```

```
LocalDate amanha = d.plusDays(1);
```

Cilada 4.7c: of() de LocalDate soma dias ← ERRADO!

of() CRIA data específica. plusDays() soma.

Java SE 7 — novidades que caem na prova

1. Classe Files (java.nio.file.Files)

Métodos estáticos para operações de arquivo: isDirectory(), createFile(), deleteIfExists(), copy(), move()

```
Files.createFile(Paths.get("/tmp/teste.txt"));
```

2. String em switch

Antes do Java 7 não era possível. A partir do Java 7:

```
switch (tipo) { case "A": ... case "B": ... }
```

3. Multicatch

Capturar múltiplos tipos de exceção em um único catch:

```
catch (IOException | SQLException e) { ... }
```

4. Literais numéricos com underscore

Para facilitar leitura de números grandes:

```
long populacao = 8_000_000_000L; // muito mais legível que 8000000000L
```

5. try-with-resources

Fecha recursos automaticamente ao sair do bloco:

```
try (BufferedReader br = new BufferedReader(new FileReader("f.txt"))) {  
    String linha = br.readLine(); // br é fechado automaticamente  
}
```

Cilada 4.5e: WatchService usa o design pattern Adapter ← ERRADO!

WatchService monitora mudanças em diretórios — não tem relação com o padrão Adapter.

Gabarito comentado — questões 4.x da prova2

Questão	Alternativa INCORRETA	Por quê está errada
4.1 Generics	d — tipo genérico mantido em runtime	Type erasure: o tipo é apagado em runtime pela JVM
4.2 HashSet	d — HashSet é ordered, LinkedHashSet unordered	É o inverso: HashSet é unordered; LinkedHashSet é ordered
4.3 Exceções	e — checked não precisam ser especificadas	Checked DEVEM ser declaradas com throws ou capturadas
4.4 ArrayList	a — ArrayList não mantém ordem pelo índice	ArrayList MANTÉM a ordem de iteração pelo índice do elemento
4.5 Java 7	e — WatchService usa padrão Adapter	WatchService não usa Adapter; monitora diretórios
4.6 Datas pré-Java8	e — Calendar herda de GregorianCalendar	É o inverso: GregorianCalendar extends Calendar
4.7 java.time	c — of() de LocalDate soma dias	of() cria data específica; para somar use plusDays()
4.8 Modelo Domínio	e — se X é número, X é classe conceitual	Se X é número ou texto no mundo real, X é ATRIBUTO
4.9 Análise/Design OO	e — projetar é mais importante que UML	UML é ferramenta essencial; as duas coisas importam